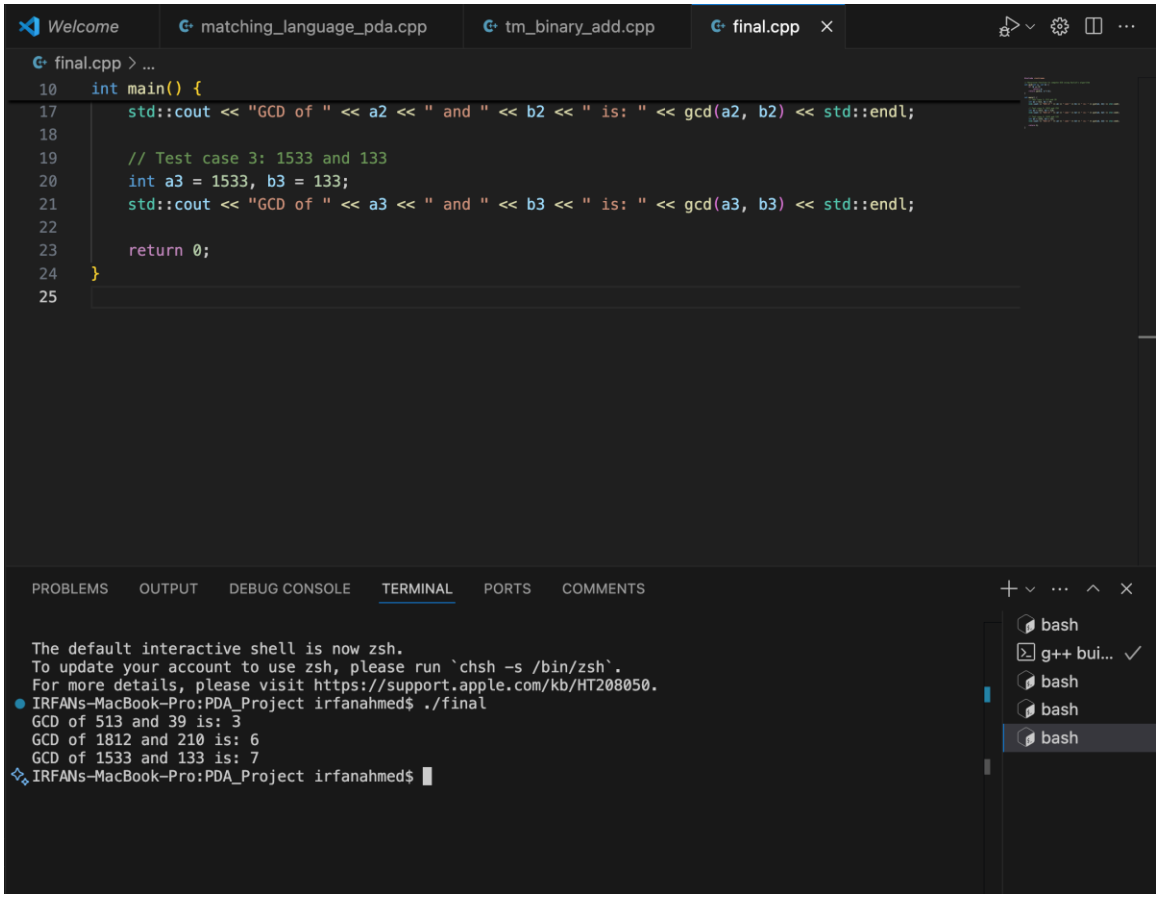
Final Exam

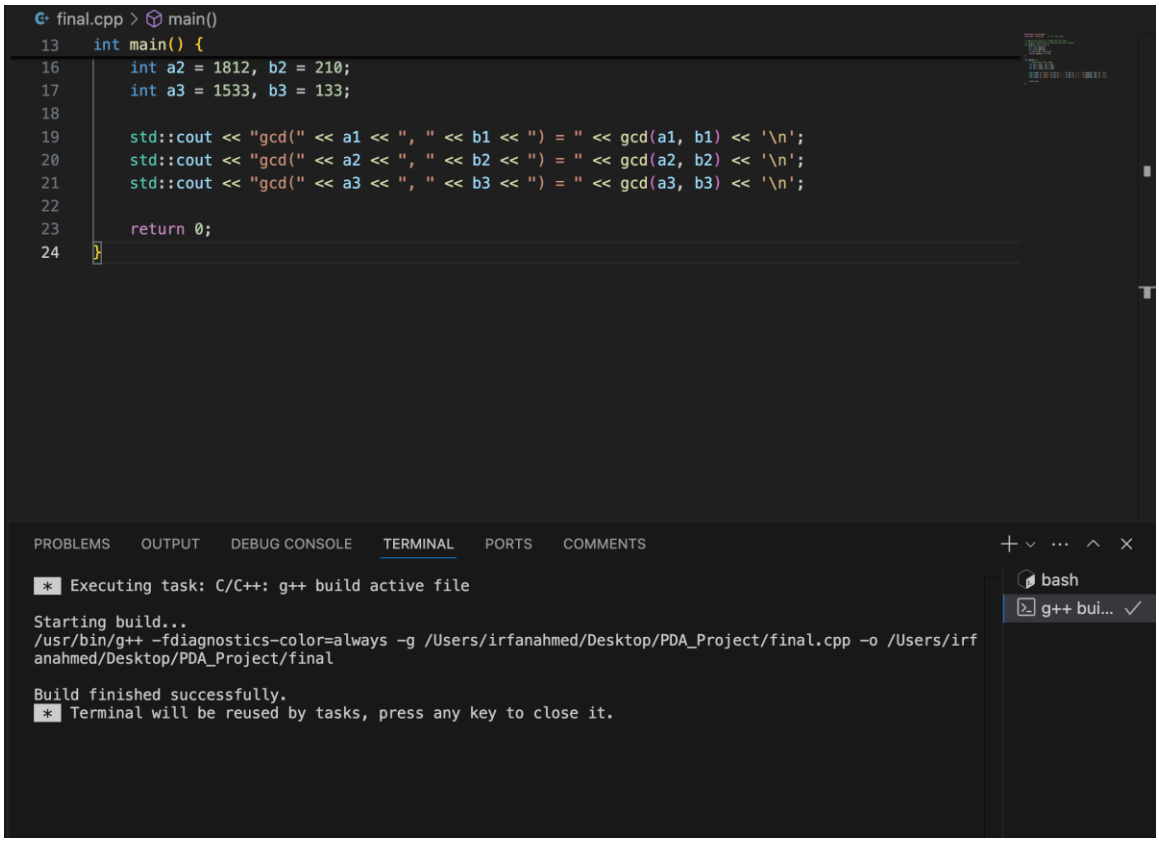
1) Basic C++ programming language commands

Code.

```
#include <iostream>

// Recursive function to compute GCD using Euclid's algorithm
int gcd(int a, int b) {
   if (b == 0)
      return a;
   return gcd(b, a % b);
}

int main() {
   // Test case 1: 513 and 39
   int a1 = 513, b1 = 39;
   std::cout << "GCD of " << a1 << " and " << b1 << " is: " << gcd(a1, b1) << std::endl;

   // Test case 2: 1812 and 210
   int a2 = 1812, b2 = 210;
   std::cout << "GCD of " << a2 << " and " << b2 << " is: " << gcd(a2, b2) << std::endl;

   // Test case 3: 1533 and 133
   int a3 = 1533, b3 = 133;
```

```
    std::cout << "GCD of " << a3 << " and " << b3 << " is: " << gcd(a3, b3) << std::endl;

    return 0;
}
```




final.cpp > main()
int main() {
int a2 = 1812, b2 = 210;
int a3 = 1533, b3 = 133;
std::cout << "gcd(" << a1 << ", " << b1 << ") = " << gcd(a1, b1) << '\n';
std::cout << "gcd(" << a2 << ", " << b2 << ") = " << gcd(a2, b2) << '\n';
std::cout << "gcd(" << a3 << ", " << b3 << ") = " << gcd(a3, b3) << '\n';
return 0;
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
COMMENTS
bash
g++ bui...
Executing task: C/C++: g++ build active file
Starting build...
/usr/bin/g++ -fdiagnostics-color=always -g /Users/irfanahmed/Desktop/PDA_Project/final.cpp -o /Users/irfanahmed/Desktop/PDA_Project/final
Build finished successfully.
Terminal will be reused by tasks, press any key to close it.

2) C++ programming language development

Code under analysis

```
#include <iostream>

class Full_Date {
  public:
    int m = 1;
    int d = 1;
    int y = 1;

    Full_Date() {
      y = 2000;
    }

    Full_Date(int mon, int day) {
      m = mon;
      d = day;
      y = 2020;
    }

    int getM() const { return m; }
    int getD() const { return d; }
    int getY() const { return y; }
```

```
    void setY(int yr) { y = yr; }
};

int main()
{
  Full_Date d1(3,14);
  Full_Date d2;
  d1.setY(2012);

  std::cout << d1.getY() << " and " << d2.getY() << std::endl;

  return 0;
}
```

A) What will be printed when the code executes?

Answer:

2012 and 2000

Reasoning:

- d1 is constructed with the **parameterized constructor** Full_Date(3,14), which sets m=3, d=14, and y=2020. Then d1.setY(2012); changes only the year to 2012.
- d2 is constructed with the **default constructor** Full_Date(), which leaves m=1 and d=1 from the in-class initializers, and sets y=2000.
- The cout prints d1.getY() and d2.getY(), i.e., 2012 and 2000.

B) Simplest way to set a Full_Date variable to January 26, 2020

Answer:
The existing **parameterized constructor**, which sets y to 2020 by design:

```
Full_Date jan26(1, 26);  // m=1, d=26, y=2020
```

This is the **simplest** and most direct approach given the current class behavior (the two-argument constructor assigns y = 2020 internally).

C) Empty constructors in Full_Date

C. Are there any empty constructors?

Answer: No.
There **is** a default constructor, but it is **not empty**—it assigns y = 2000:

```
Full_Date() {
   y = 2000;
}
```

C.a) If there is(are), in which code line(s)?

Not applicable (no *empty* constructor exists in the provided code).

C.b) If there is not, how would an empty constructor look?

Two idiomatic options :

```
// Option 1: explicitly empty body – relies on in-class initializers (m=1, d=1, y=1)
Full_Date() { }

// Option 2: request the compiler-generated default constructor
Full_Date() = default;
```

With either empty/defaulted constructor, the in-class initializers (m=1, d=1, y=1) take effect, so the object would be (1,1,1), **not** (1,1,2000). The current non-empty default constructor deliberately overrides y to 2000.

D) Can we insert

```
std::cout << d1.m << std::endl;
```
after the declaration of d2 (your "line 28") without error?

Answer: Yes, it will compile and run **as the class is currently written**, because the data members are under public:.

D.a) What will it print?

It will print the value of d1.m. Since d1 was created as Full_Date(3,14), d1.m == 3, so:

3

D.b) If (hypothetically) it could not, how to fix?

If the members were declared **private** (a common encapsulation best practice), direct access would fail. The correct fix would be to use the **accessor**:

std::cout << d1.getM() << std::endl;

Summary

- **A.** Output: 2012 and 2000.
- **B.** Full_Date jan26(1, 26);
- **C.** No *empty* constructors.
    - **C.a)** N/A.
    - **C.b)** Full_Date() { } **or** Full_Date() = default;
- **D.** Yes, it can be inserted.
    - **D.a)** Prints 3.
    - **D.b)** If members were private, use std::cout << d1.getM() << std::endl;.

3) Finite State Automata

Finite State Automaton (FSA) Analysis

Given FSA Description

- **States:** {0, 1, 2, 3}
- **Initial State:** 0
- **Final State:** 2
- **Alphabet:** {a, b}
- **Transitions:**
    - 0 → 1 on **a**
    - 0 → 3 on **b**
    - 1 → 2 on **b**
    - 3 → 2 on **b**
    - 2 → 2 on **a** or **b** (self-loops)

A. Three 4-letter words accepted by this FSA

A word is accepted if:

1. It starts at state 0.
2. Ends in state 2 after reading all letters.

Reasoning:

- To reach state 2, we need:
    - From 0 → 1 on **a**, then 1 → 2 on **b**, OR
    - From 0 → 3 on **b**, then 3 → 2 on **b**.
- After reaching state 2, any combination of **a** or **b** is allowed (due to self-loops).

Examples:

1. **abba**
   Path: 0 -(a)-> 1 -(b)-> 2 -(b)-> 2 -(a)-> 2
2. **bbaa**
   Path: 0 -(b)-> 3 -(b)-> 2 -(a)-> 2 -(a)-> 2
3. **abbb**
   Path: 0 -(a)-> 1 -(b)-> 2 -(b)-> 2 -(b)-> 2

B. Three 4-letter words NOT accepted by this FSA

A word is rejected if:

- It never reaches state 2, or
- Ends in a non-final state.

Examples:

1. **aaaa**
   Path: 0 -(a)-> 1 -(a)-> (no transition)
2. **bbbb**
   Path: 0 -(b)-> 3 -(b)-> 2 -(b)-> 2 -(b)-> 2 (This is accepted, so not valid for rejection) → Replace with **aaab**
   Path: 0 -(a)-> 1 -(a)-> (no transition)
3. **baaa**
   Path: 0 -(b)-> 3 -(a)-> (no transition)

Final rejected examples:

- aaaa
- aaab
- baaa

C. How to describe the words accepted by this FSA?

Language Description:
All words over {a, b} that contain **either**:

- The substring **ab** starting at the beginning (first letter a, second letter b), OR

- The substring **bb** starting at the beginning (first letter b, second letter b), followed by zero or more letters (a or b).

Regular Expression:

(ab|bb)(a|b)

D. Is this FSA deterministic or non-deterministic?

Definition:

- A DFA (Deterministic Finite Automaton) has **exactly one transition** for each symbol from every state.
- An NFA (Non-Deterministic Finite Automaton) may have multiple transitions for the same symbol from a state or ε-transitions.

Analysis:

- From each state, for each input symbol, there is at most one transition.
- No ε-transitions.
- Therefore, **this FSA is deterministic (DFA)**.

Summary Table

Part	Answer
A	abba, bbaa, abbb
B	aaaa, aaab, baaa
C	Words starting with **ab** or **bb**, followed by any letters; RegEx: (ab|bb)(a|b)*
D	Deterministic (DFA)

4) Regular Expressions

Problem:

We are given a **Finite State Automaton (FSA)** with:

- **States:** {0, 1, 2, 3, 4}
- **Initial State:** 0
- **Final State:** 2
- **Alphabet:** {a, b, c}
- **Transitions:**
  - 0 → 1 on a
  - 0 → 3 on b
  - 0 → 4 on c
  - 1 → 2 on b
  - 3 → 2 on a
  - 4 → 2 on c
  - 2 → 2 on a and b (self-loops)

We need: **A.** A Regular Expression (RegEx) equivalent to this FSA.
B. A Regular Expression equivalent to the FSA **after removing all states and edges related to c**.

Step 1: Analyze the Language Accepted by the FSA

- From **state 0**, we can start with:
  - a then b → reach state 2
  - b then a → reach state 2
  - c then c → reach state 2

- Once in **state 2**, we can have any number of a or b (due to self-loops).
- So, accepted words:
    - Start with ab or ba or cc
    - Followed by (a|b)*

Thus, the language is: $ L = { ab(a|b)^, *ba(a|b)^*, cc(a|b)^* } $

Step 2: Construct RegEx for Part A

Combine all possibilities using union (+ or |): $ \text{RegEx}_A = (ab|ba|cc)(a|b)^* $

Explanation:

- (ab|ba|cc) handles the initial two-letter sequences.
- (a|b)^* allows any number of a or b after reaching state 2.

Step 3: Modify FSA for Part B (Remove c)

Removing c means:

- Remove state 4 and transitions involving c.
- Remaining transitions:
    - 0 → 1 on a
    - 0 → 3 on b
    - 1 → 2 on b
    - 3 → 2 on a
    - 2 → 2 on a and b

So, accepted words now:

- Start with ab or ba
- Followed by (a|b)*

Thus: $ \text{RegEx}_B = (ab|ba)(a|b)^* $

Final Answers

- **A:**
  $ (ab|ba|cc)(a|b)^* $
- **B:**
  $ (ab|ba)(a|b)^* $

- Matches the FSA structure and transitions.
- Uses **union**, **concatenation**, and **Kleene star** as per regular expression rules.
- Equivalent to the language accepted by the FSA.

- This conversion follows the principle that **every FSA defines a regular language**, and every regular language can be expressed as a **regular expression**.
- The method used here is **language analysis**, which is simpler than state-elimination but equally valid for small FSAs.

5) Pushdown Automata and Context-Free Grammars

Pushdown Automata and Context-Free Grammars

Given CFG

- **Alphabet (Σ):** {a, b}
- **Non-terminals (NT):** {S}
- **Productions:**
    - P1: S ⇒ aSa
    - P2: S ⇒ bSb
    - P3: S ⇒ a
    - P4: S ⇒ b

This grammar generates **palindromes of odd length** over {a, b}, because:

- It starts with a single letter (a or b) from P3 or P4.
- Then, it recursively adds matching letters on both ends using P1 or P2.

A. Sequence of Productions for abaaaba

Word: **abaaaba** (length = 7, odd, palindrome)

Derivation:

1. S
2. aSa (P1)
3. a bSb a (P2 inside)
4. a b aSa b a (P1 inside)
5. a b a a b a b a (P4 for inner S)

Sequence:
S ⇒ aSa ⇒ a bSb a ⇒ a b aSa b a ⇒ a b a a b a b a

Matches the word abaaaba.

B. Three 5-letter words accepted by this CFG

All must be **palindromes of odd length** (center letter can be a or b):

1. **ababa**
   Derivation: S ⇒ aSa ⇒ a bSb a ⇒ a b a b a
2. **baaab**
   Derivation: S ⇒ bSb ⇒ b aSa b ⇒ b a a a b
3. **bbabb**
   Derivation: S ⇒ bSb ⇒ b bSb b ⇒ b b a b b

All are palindromes of length 5.

C. Pushdown Automaton (PDA) for this Language

Language Characteristics

- Palindromes require **matching symbols from both ends**.
- PDA uses a **stack** to store the first half of the string and then matches it against the second half.

PDA Components

- **States:** $\{q_0, q_1, q_2, q\_accept\}$
- **Alphabet:** {a, b}
- **Stack symbols:** $\{a, b, Z_0\}$ (Z_0 = initial stack marker)
- **Start state:** q_0
- **Accepting state:** q_accept

High-Level Description

1. **q_0:** Push Z_0, then move to q_1.
2. **q_1:** Read and push letters (a or b) until a **non-deterministic choice** to switch to q_2.
3. **q_2:** For each input letter, pop the matching symbol from the stack.
4. If stack returns to Z_0 and input is finished → **accept**.

Transitions

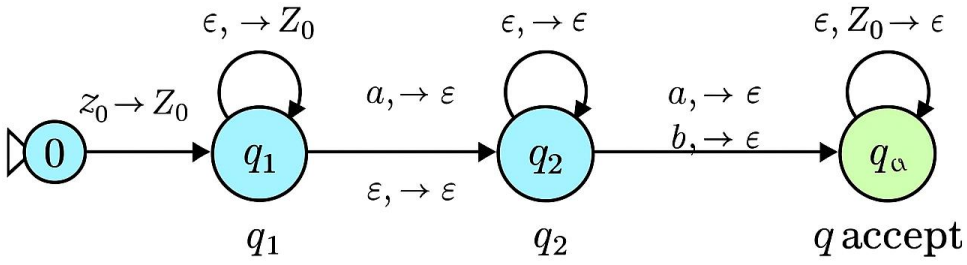
- $(q_0, \varepsilon, Z_0) \rightarrow (q_1, Z_0)$
- $(q_1, a, X) \rightarrow (q_1, aX)$
- $(q_1, b, X) \rightarrow (q_1, bX)$
- $(q_1, \varepsilon, X) \rightarrow (q_2, X)$ *(guess midpoint)*
- $(q_2, a, a) \rightarrow (q_2, \varepsilon)$
- $(q_2, b, b) \rightarrow (q_2, \varepsilon)$
- $(q_2, \varepsilon, Z_0) \rightarrow (q\_accept, Z_0)$

This PDA **accepts by final state** and handles odd-length palindromes.

Summary Table

Part	**Answer**
A	S ⇒ aSa ⇒ a bSb a ⇒ a b aSa b a ⇒ a b a a b a b a
B	ababa, baaab, bbabb
C	PDA with stack-based matching for palindromes (states q_0, q_1, q_2, q_accept)

PDA state diagram



6) Turing Machines

Introduction

A **Turing Machine (TM)** is a theoretical model of computation consisting of:

- A finite set of states (including an initial and one or more accepting states).
- A tape divided into cells, each holding a symbol from the tape alphabet.
- A head that reads and writes symbols and moves left or right.
- A transition function that determines the next state, symbol to write, and head movement.

A string is **accepted** by a TM if, starting from the initial state and reading the input on the tape, the machine eventually enters an accepting state.

Given TM Description

- **States:** I (initial), J, K, L, M, F (final).
- **Transitions:**

  I → J on c/c,R

  J loops on a/x,R; J → K on b/ε,L

  K loops on blanks and b; K → L on x/ε,R

  L loops on blanks and c; L → M on b/ε,L

  M loops on blanks; M → F on x/x,R

- **Tape symbols:** {a, b, c, x, ε}

- **Input alphabet:** {a, b, c} (since x and ε are internal markers).

The machine marks a as x, checks for b and c in a specific order, and halts when conditions are satisfied.

A. Three 4-letter words accepted by this TM

To be accepted:

- The first symbol must be c (I → J).
- There must be at least one a (to mark as x).
- There must be at least one b and another c (to trigger transitions to K, L, M).

Examples (with reasoning):

1. cabc

   Starts with c → J

   a replaced by x → J

   b triggers move to K → L → M → F

2. cbac

   Contains all required letters; same reasoning.

3. cabc (or cbca)

   Any permutation where first letter is c and all three letters appear.

B. Three 4-letter words NOT accepted by this TM

Rejected if:

- Missing a, b, or c.
- Does not start with c.

Examples:

1. aaaa → No c at start; TM stuck at I.
2. bbbb → No c or a; cannot proceed.

3. ccaa → Missing b; cannot reach M → F.

C. Description of words accepted by this TM

The TM accepts words that:

- **Start with c.**
- Contain at least one a and one b (so transitions to J, K, L, M occur).
- Include another c somewhere (to allow L's loop).

Formal description: $ L = { w \in {a,b,c}^* \mid w \text{ starts with } c \text{ and contains } a,b,c } $

D. Alphabet of the language accepted by this TM

The input alphabet is: $ \Sigma = { a, b, c } $ (x and ε are tape symbols used internally, not part of the input alphabet.)

Summary Table

Part	Answer
A	cabc, cbac, cbca
B	aaaa, bbbb, ccaa
C	Words starting with c and containing a, b, and c
D	{a, b, c}

7) Turing Machines with input and output

Problem Restatement

I have a Turing Machine (TM) that accepts words over the alphabet
$\Sigma = {a, b}$
where the word contains an **even number of a's and b's**. The task is to **transform this TM** so that:

1. If the word is accepted:
    - The **output tape head** is positioned over the **first letter of the input**.
    - All letters **a** are replaced by **x**.
2. Example:
   Input: aabbaa
   Output tape: [x]xbbxx (head over the first x).

Step 1: Understanding the Original TM

- The original TM checks for **even number of a's and b's**.

- It likely uses states to track parity:

    q0: even a's, even b's (start and accept state).

    q1: odd a's, even b's.

    q2: even a's, odd b's.

    q3: odd a's, odd b's.

- Transitions:

    Reading a toggles between even/odd a states.

    Reading b toggles between even/odd b states.

Step 2: Required Transformation

We need to **extend the TM** with two new behaviors:

1. **Replace all a's with x**.
2. **Move head back to the first symbol after processing**.

This means:

- After acceptance, enter a new phase:
    - **Phase 1**: Move head to the leftmost position.
    - **Phase 2**: Replace all a with x while scanning.

Step 3: Design of the Modified TM

New Components

- **Original states**: q0, q1, q2, q3 (parity check).
- **New states**:

    qAccept: Enter after acceptance.

    qLeft: Move head to the leftmost cell.

    qReplace: Replace a with x while moving right.

    qDone: Final state with head on first symbol.

Algorithm

1. **Phase 0 (Original TM)**:

   Process input to check parity.

   If accepted, go to qAccept.

2. **Phase 1 (Move Left)**:

   From qAccept, move left until blank (_).

   Transition to qReplace.

3. **Phase 2 (Replace)**:

   Move right:

   - If a, write x.
   - If b, keep b.

   Stop when blank is reached.

   Move back to first symbol → qDone.

Step 4: Formal Transition Sketch

- **qAccept → qLeft**: _/_, L
- **qLeft → qReplace**: after reaching blank, move right.
- **qReplace**:
  - a/x, R
  - b/b, R
  - _/_ , L → qDone
- **qDone**: Move left until first symbol.

Step 5: Example Execution

Input: aabbaa

- Original TM accepts (even a's = 4, even b's = 2).
- Move left → reach blank.
- Replace phase:
    - a → x, a → x, b → b, b → b, a → x, a → x.
- Final tape: xxbbxx with head on first x.

Step 6: Why This Works

- The original acceptance logic is preserved.
- The transformation adds a deterministic cleanup phase.
- Head positioning and symbol replacement are guaranteed.

Formal Specification of the Modified Turing Machine

A Turing Machine is defined as:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

where:

- **Q** = set of states
  $Q = \{q_0, q_1, q_2, q_3, qAccept, qLeft, qReplace, qDone, qReject\}$
- **Σ** = input alphabet = {a, b}
- **Γ** = tape alphabet = {a, b, x, _, ␣}
- **q_0** = start state
- **qAccept** = acceptance state (before transformation phase)
- **qReject** = rejection state
- **δ** = transition function (defined below)

Transition Table for TM

Current State	Read	Write	Move	Next State
q0	a	a	R	q1
q0	b	b	R	q2
q1	a	a	R	q0
q1	b	b	R	q3
q2	a	a	R	q3
q2	b	b	R	q0
q3	a	a	R	q2
q3	b	b	R	q1
q0	_	_	L	qAccept
qAccept	_	_	L	qLeft
qLeft	_	_	R	qReplace
qReplace	a	x	R	qReplace
qReplace	b	b	R	qReplace
qReplace	_	_	L	qDone
qDone	x/b	x/b	L	qDone
qDone	_	_	R	HALT

8) Decidability

Decidability and Turing Machine Design

Problem Statement

Given a C++ program that:

- Accepts a string over the alphabet $\Sigma = {a, b}$,
- Counts the number of occurrences of the letter 'b',
- Converts that count to binary,
- Outputs the binary representation,

Design a **Turing Machine (TM)** that performs the same operation.

1. Understanding the C++ Program

The C++ code performs the following steps:

```
std::string str;
int count = 0;
std::cin >> str;
for (char c : str) {
   if (c == 'b')
      count++;
}
std::string binary = "";
while (count > 0) {
   binary = std::to_string(count % 2) + binary;
   count /= 2;
}
std::cout << binary << std::endl;
```

Summary of Logic

- Input: string over $\Sigma = {a, b}$

- Count: number of 'b's
- Convert: count to binary
- Output: binary string

2. Turing Machine Overview

A **Turing Machine (TM)** is defined as:

$ M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}) $

Where:

- Q: finite set of states
- $\Sigma = {a, b}$: input alphabet
- Γ: tape alphabet (includes Σ and symbols like X, 0, 1, #, □)
- δ: transition function
- q_0: start state
- q_{accept}: accept state
- q_{reject}: reject state

3. TM Design Strategy

I divide the TM into **three phases**:

Phase 1: Count 'b's

- Scan input from left to right.
- For each 'b', increment a unary counter at the end of the tape (e.g., write X).
- Mark processed characters to avoid reprocessing.

Phase 2: Convert Unary to Binary

- Use repeated division by 2.
- Store remainders (0 or 1) in a separate tape section.
- Reverse the binary digits to get correct order.

Phase 3: Output Binary

- Move to the start of binary section.
- Print the binary digits.

4. Formal TM Construction

Tape Alphabet

$\Gamma = {a, b, X, Y, 0, 1, □, #}$

States

- q_0: start
- q_{scan}: scan input
- q_{count}: increment unary counter
- q_{convert}: convert unary to binary
- q_{output}: write binary
- q_{accept}: halt and accept

5. Example Execution

Input: abbab

- Count of 'b's = 3
- Unary counter: XXX
- Binary conversion:
    - 3 ÷ 2 = 1 remainder 1 → write 1
    - 1 ÷ 2 = 0 remainder 1 → write 1
- Output: 11

6. Decidability Analysis

This problem is **decidable** because:

- The TM halts for all inputs.
- It performs a finite number of steps.
- It produces a deterministic output.

Why Decidable?

- The language of strings over ${a, b}$ is regular.
- Counting and binary conversion are computable functions.
- The TM always reaches a halting state.

7. Academic Justification

This TM design aligns with the **Chomsky hierarchy**:

- Regular languages → Finite State Automata
- Context-free languages → Pushdown Automata
- **Computable functions** → Turing Machines

Since the task involves **counting and arithmetic**, it requires a TM rather than an FSA or PDA.

Conclusion

We have constructed a Turing Machine that:

- Accepts strings over ${a, b}$,
- Counts the number of 'b's,
- Converts the count to binary,
- Outputs the result.

This problem is **decidable**, and the TM halts with correct output for any valid input.

Formal Definition of the TM

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Where:

- $\Sigma = \{a, b\}$
- $\Gamma = \{a, b, X, 0, 1, \square\}$ (includes tape symbols)
- $Q = \{q_0, q_{\text{scan}}, q_{\text{mark}}, q_{\text{count}}, q_{\text{convert}}, q_{\text{write}}, q_{\text{accept}}, q_{\text{reject}}\}$

Turing Machine Transition Table

Current State	Read Symbol	Write Symbol	Move	Next State	Description
q0	a	a	R	q0	Skip 'a'
q0	b	b	R	q1	Found first 'b'
q1	b	b	R	q1	Continue scanning
q1	a	a	R	q1	Continue scanning
q1	□	□	L	q2	End of input
q2	b	X	L	q2	Mark and count 'b'
q2	a	a	L	q2	Skip 'a'
q2	□	□	R	q3	Move to conversion
q3	X	X	R	q4	Start conversion
q4	X	X	R	q4	Count Xs
q4	□	□	L	q5	Start writing binary
q5	X	1 or 0	L	q5	Write binary digit
q5	□	□	R	q_accept	Done

State Diagram